

# Discussion 1

Introduction to Socket Programming  
&  
A1 Introduction

# Outline

- Logistics
- A1 Intro
- Introduction to socket programming

# Logistics

---

# Who am I?

- I'm Efe!
- Finished my undergrad last year, finishing up SUGS this year
- International student from Turkey 🇹🇷, living in Ann Arbor since high school
- IA for 281 for 4 semesters, second semester as a GSI for 489
- Moving to Miami for job next year! 🌴
- Ask me about
  - IA / GSI
  - International student things: (OPT/CPT)
  - Job stuff



Figure 1: Me

- Course website on [GitHub](#)
- Complete VM setup ASAP!
  - Ubuntu instances on the VM will also be used for A4!
- A1 is out! Due **Monday, Jan. 29**
  - A1 focuses on creating a tool similar to iPerf, which measures the speed of a network connection.
  - All assignments are submitted through [the Autograder](#)
  - A1 is individual, while the rest can be completed in groups of 2-3 people.

# Assignment 1

## Introduction

---

- For assignment 1, you will be using a virtual machine (VM)
- There is a VM setup guide in the same repo as the project
- VM setup is a time commitment of anywhere from 30 minutes to an hour!
  - Get started as soon as possible!

- In assignment 1, you will be implementing a program that measures the bandwidth and latency of sending data over TCP in a network
- You will use socket programming to achieve this task
  - iPerfer server: waits for a client to connect, receives data, and sends ACKs<sup>1</sup>
  - iPerfer client: sends data to a iPerfer client
- Both the server and client measure the observed rate of data and latency; display statistics at the end

---

<sup>1</sup>These are application level ACKs, not TCP ACKs that are used within the protocol



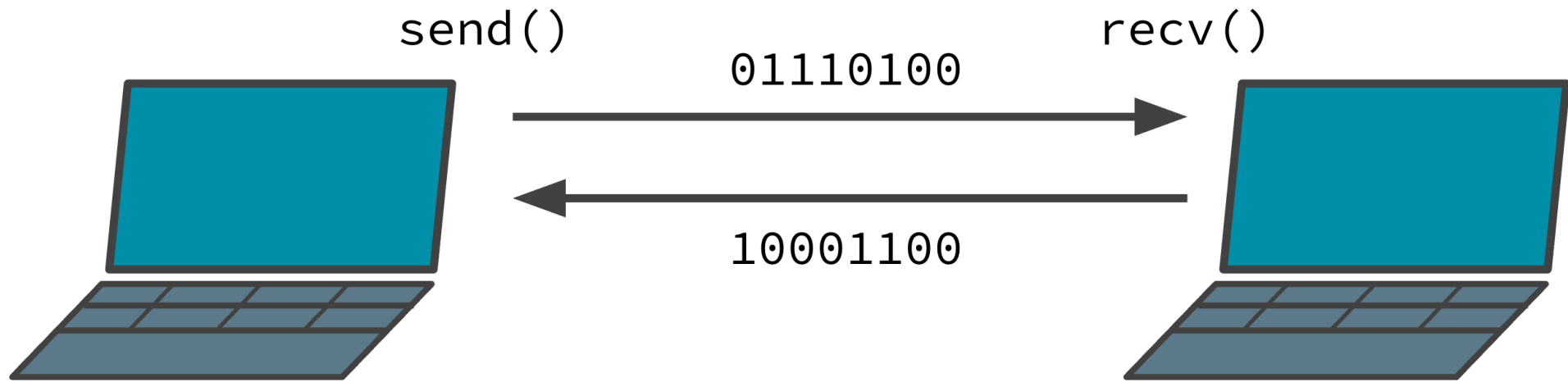
- Important links:
  - Spec: <https://github.com/eecs489staff/a1-sockets-mininet>
  - Introduction to Modern CMake: <https://cliutils.gitlab.io/modern-cmake/chapters/basics.html>
  - Beej's Guide to Network Programming: <https://beej.us/guide/bgnet/>

# Intro to Socket Programming

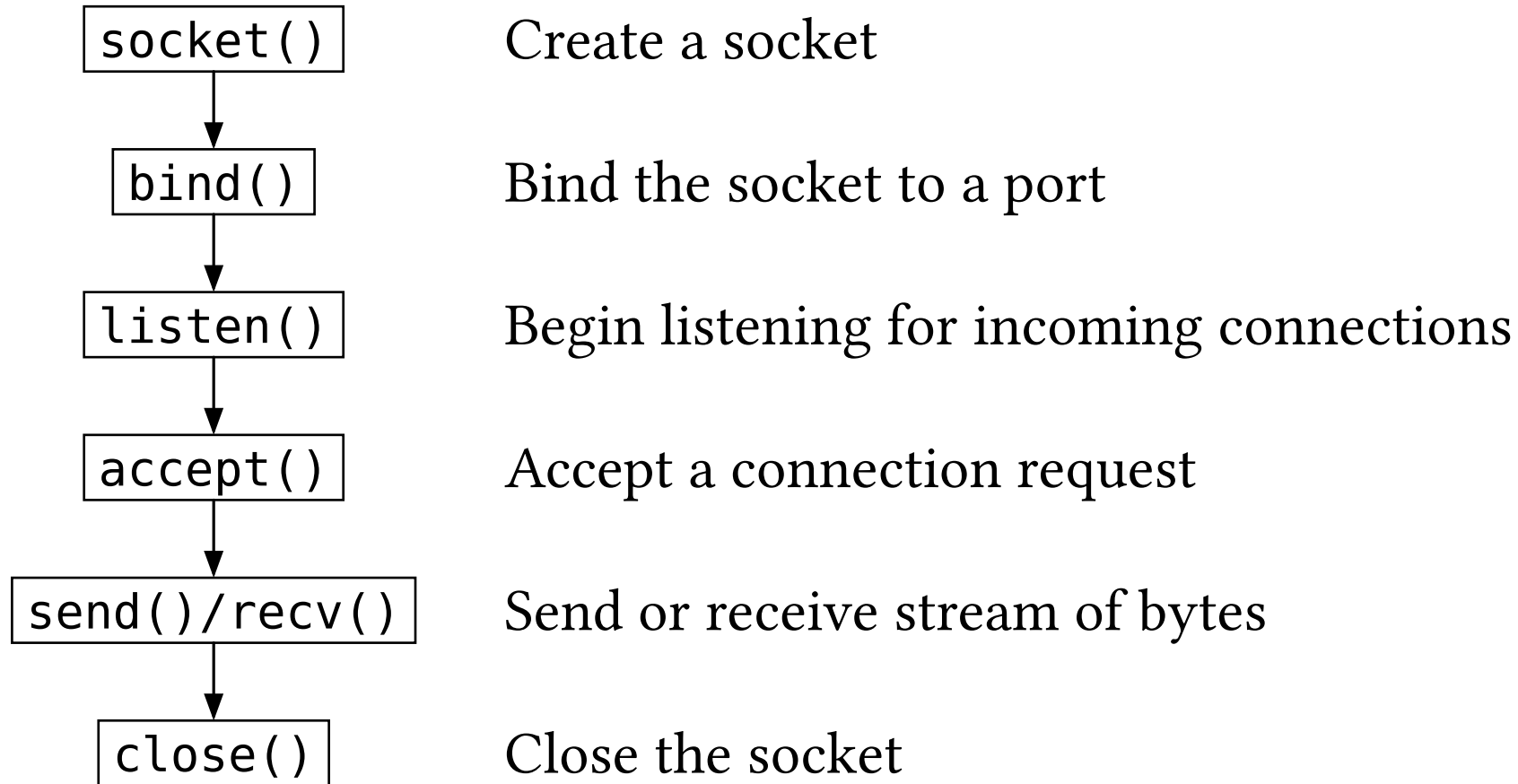
---

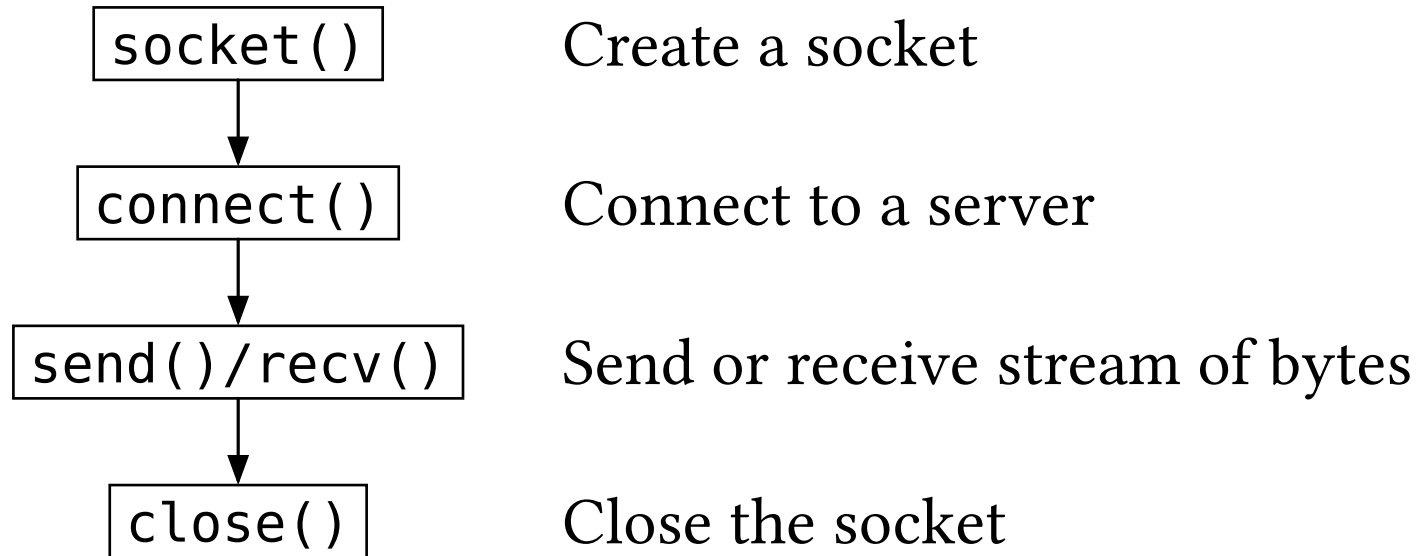
# What is a Socket?

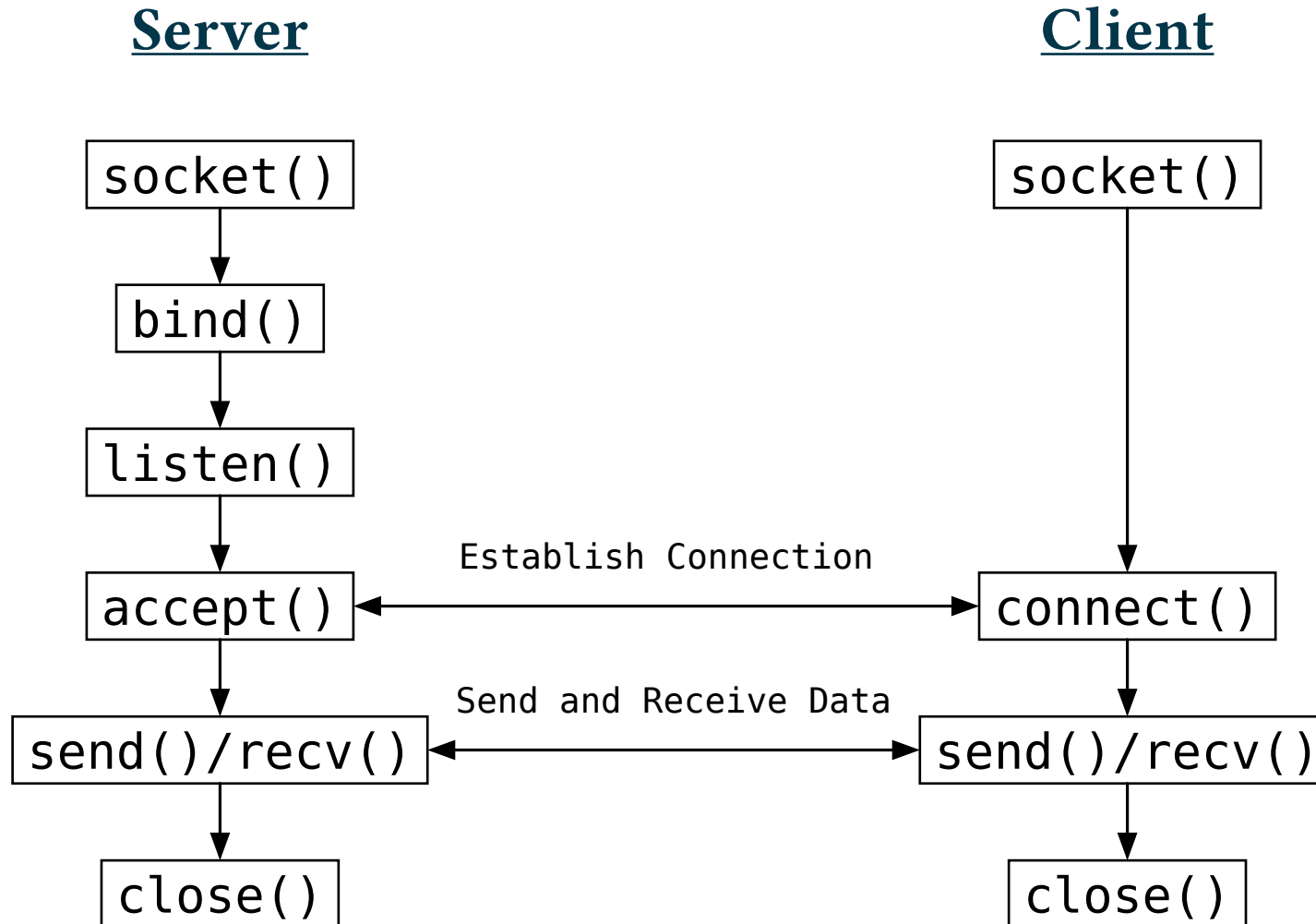
- A socket is a two-way communication channel between two processes or machines
- Sockets abstract away many parts of communicating over a network
  - Provides a programming interface (API) to exchange data
  - Hides details of interacting with network card



```
1  int socket(domain, type, protocol);
2
3  int send(sockfd, buffer, size, flags);
4  int recv(sockfd, buffer, size, flags);
5
6  int bind    (sockfd, addr);
7  int listen  (sockfd, backlog);
8  int accept  (sockfd, 0, 0);
9  int connect (sockfd, addr);
10 int close   (sockfd);
```







```
socket(int domain, int type, int protocol);
```

- **domain:** Specifies the communication domain; AF\_INET is for IPv4 protocol
- **type:** Specifies the communication semantics; for a two-way communication socket, we use SOCK\_STREAM
- **protocol:** Specifies the protocol; this will be IPPROTO\_TCP for our purposes

*Note: These symbols are defined in sys/socket.h*

```
1 // ...
2 int sockfd = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
3 ssize_t bytes_sent = send(sockfd, my_vec.data(), my_vec.size(), 0);
```



```
int bind(int sockfd, const struct sockaddr_in *addr,  
        socklen_t addrlen);
```

Binds the socket to a specific port, among other things.

- **sockfd**: The socket file descriptor from `socket()`
- **addr**: A `sockaddr` struct that specifies some parameters, including which port to be used (via `addr.sin_port`)
  - If this is set to 0, the OS chooses a port for you
  - The `sockaddr` type is a generalization of `sockaddr_in`, which is the internet type
- **addrlen**: `sizeof(sockaddr)`

Returns 0 on success, -1 on error.

# Aside: What's a Port?

- A port is a logical channel (number) used alongside an IP address to route data to specific services or applications

The well-known ports (also known as *system ports*) are those numbered from 0 through 1023. The requirements for new assignments in this range are stricter than for other registrations.<sup>[2]</sup>

Notable well-known port numbers

| Number   | Assignment                                                                                    |
|----------|-----------------------------------------------------------------------------------------------|
| 20       | <a href="#">File Transfer Protocol</a> (FTP) Data Transfer                                    |
| 21       | <a href="#">File Transfer Protocol</a> (FTP) Command Control                                  |
| 22       | <a href="#">Secure Shell</a> (SSH) Secure Login                                               |
| 23       | <a href="#">Telnet</a> remote login service, unencrypted text messages                        |
| 25       | <a href="#">Simple Mail Transfer Protocol</a> (SMTP) email delivery                           |
| 53       | <a href="#">Domain Name System</a> (DNS) service                                              |
| 67, 68   | <a href="#">Dynamic Host Configuration Protocol</a> (DHCP)                                    |
| 80       | <a href="#">Hypertext Transfer Protocol</a> (HTTP) used in the <a href="#">World Wide Web</a> |
| 110      | <a href="#">Post Office Protocol</a> (POP3)                                                   |
| 119      | <a href="#">Network News Transfer Protocol</a> (NNTP)                                         |
| 123      | <a href="#">Network Time Protocol</a> (NTP)                                                   |
| 143      | <a href="#">Internet Message Access Protocol</a> (IMAP) Management of digital mail            |
| 161      | <a href="#">Simple Network Management Protocol</a> (SNMP)                                     |
| 194      | <a href="#">Internet Relay Chat</a> (IRC)                                                     |
| 443      | <a href="#">HTTP Secure</a> (HTTPS) HTTP over TLS/SSL                                         |
| 546, 547 | <a href="#">DHCPv6</a> IPv6 version of DHCP                                                   |

The registered ports are those from 1024 through 49151. IANA maintains the official list of well-known and registered ranges.<sup>[3]</sup>

The dynamic or private ports are those from 49152 through 65535. One common use for this range is for [ephemeral ports](#).

# Aside: sockaddr\_in

## Intro to Socket Programming

```
1  int make_server_sockaddr(struct sockaddr_in *addr,
2      int port) {
3      // Step (1): specify socket family.
4      // This is an internet socket.
5      addr->sin_family = AF_INET;
6      // Step (2): specify socket address(hostname).
7      // The socket will be a server, so it will only be
8      // listening.
9      // Let the OS map it to the correct address.
10     addr->sin_addr.s_addr = INADDR_ANY;
11     // Step (3): Set the port value.
12     // If port is 0, the OS will choose the port for us.
13     // Use htons to convert from local byte order to
14     // network byte
15     addr->sin_port = htons(port);
16     return 0;
17 }
```

```
1  int make_client_sockaddr(struct sockaddr_in addr,
2      const char *hostname, int port) {
3      // Step (1): specify socket family.
4      // This is an internet socket.
5      addr->sin_family = AF_INET;
6      // Step (2): specify socket address (hostname).
7      // The socket will be a client, so call this unix
8      // helper function
9      // to convert a hostname string to a usable 'hostent'
10     struct.
11     struct hostent *host = gethostbyname(hostname);
12     if (host == NULL) {
13         printf(stderr, "%s: unknown host\n", hostname);
14         return -1;
15     }
16     memcpy(&addr->sin_addr, host->h_addr, host->h_length);
17     // Step (3): Set the port value.
18     // Use htons to convert from local byte order to
19     // network byte order.
20     addr->sin_port = htons(port);
21     return 0;
22 }
```

- Suppose that a function like `bind()` or `send()` returns `-1`, indicating an error
  - How do we know what went wrong?
- Calls such as these will set a global<sup>1</sup> `errno` variable indicating what went wrong
  - `errno` can be found in `<errno.h>`
  - `errno` is just a number, but functions like `strerror` output a meaningful code

```
1 int success = bind(sockfd, &bind_addr, sizeof(bind_addr));
2 if (success == -1) {
3     spdlog::error("Error while binding to socket: {}", strerror(errno));
4 }
```

---

<sup>1</sup>Actually thread-local, not global

```
int listen(int sockfd, int backlog);
```

- **sockfd**: The socket to mark as being used to accept incoming connections
- **backlog**: The maximum length the queue of connections waiting to be accepted can grow to

```
int connect(int sockfd, const struct sockaddr_in *addr,  
            socklen_t addrlen);
```

- **sockfd**: The socket file descriptor from `socket()`
- **addr**: A `sockaddr_in` struct specifying the server's IP address and port
- **addrlen**: The size of the `sockaddr_in` struct

Attempts to establish a connection to the server. Returns 0 on success, -1 on error.

```
ssize_t send(int sockfd, const void *buf, size_t len, int flags);
```

- **sockfd**: The socket file descriptor used to send data
- **buf**: A pointer to the data to send
- **len**: The number of bytes to send
- **flags**: Additional options for sending; typically 0 for standard behavior

Returns the number of bytes sent, or -1 on error.

# send() — Example

```
1  std::vector<char> message{'H', 'e', 'l', 'l', 'o', '!', '\0'};
2  size_t total_bytes_sent = 0;
3
4  while (total_bytes_sent < message.size()) {
5      ssize_t bytes_sent = send(sockfd,
6          message.data() + total_bytes_sent,
7          message.size() - total_bytes_sent, 0);
8
9      if (bytes_sent == -1) {
10         spdlog::error("Send error: {}", strerror(errno));
11         break; // Exit the loop on error
12     }
13     total_bytes_sent += bytes_sent; // Update the count of sent bytes
14 }
```



```
ssize_t recv(int sockfd, void *buf, size_t len, int flags);
```

- **sockfd**: The socket file descriptor used to receive data
- **buf**: A pointer to the buffer where received data will be stored
- **len**: The maximum number of bytes to read
- **flags**: Options for receiving; commonly 0 for standard behavior
  - Useful: MSG\_WAITALL will wait to receive len bytes before returning

Returns the number of bytes received, 0 for end-of-stream (connection closed), or -1 on error.

```
1 char buffer[1024];  
2 ssize_t bytes_received = recv(sockfd, buffer, sizeof(buffer), 0);
```

- sizeof is a common source of errors in 489
- What does the following code print?

```
1  std::vector<char> my_vec;  
2  for (size_t i = 0; i < 100; ++i) {  
3      my_vec.push_back('a');  
4  }  
5  
6  spdlog::info("Sizeof my_vec: {}", sizeof(my_vec))
```

- A) 100
- B) 4
- C) 8
- D) Can't be determined by the code alone

- sizeof is a common source of errors in 489
- What does the following code print?

```
1  std::vector<char> my_vec;  
2  for (size_t i = 0; i < 100; ++i) {  
3      my_vec.push_back('a');  
4  }  
5  
6  spdlog::info("Sizeof my_vec: {}", sizeof(my_vec))
```

- A) 100
- B) 4
- C) 8
- **D) Can't be determined by the code alone**

- sizeof returns the size of a type
- For example, consider

```
1 struct MyStruct {  
2     int a;  
3     int b;  
4 }
```

- ▶ sizeof(MyStruct) == 8, since it contains two ints
- A std::vector contains a pointer to some data, plus some bookkeeping variables for keeping track of its size
  - ▶ This means that sizeof(std::vector) is equal to the size of the bookkeeping variables + the size of a pointer (4 bytes on 32-bit systems, 8 bytes on 64-bit systems).

```
int close(int sockfd);
```

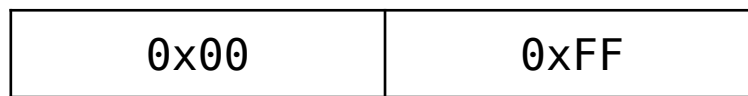
- **sockfd**: The socket file descriptor to close

Closes the socket and releases associated resources. Returns 0 on success, -1 on error.

```
1 if (close(sockfd) == -1) {  
2     spdlog::error("Close error: {}", strerror(errno));  
3 } else {  
4     spdlog::info("Socket successfully closed");  
5 }
```

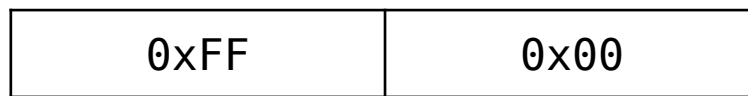
# Note - Byte Order

- When storing numbers, different computers order the most-significant bytes in different ways
- For example, consider the two-byte integer 65280 (0xFF00).
  - In a *little-endian* system, the least significant byte is stored at the lowest address



addresses grow →

- In a *big-endian* system, the most significant byte is stored at the lowest address



addresses grow →

# Note - Byte Order

- What happens if a little-endian system sends a number to a big-endian system?
  - send/recv just send bytes in the order they're in memory
- To avoid this issue, we always convert numbers to *network-order* (big-endian) before sending them over networks

| Function | Meaning                                               |
|----------|-------------------------------------------------------|
| ntohl    | <b>n</b> etwork <b>t</b> o <b>h</b> ost <b>l</b> ong  |
| ntohs    | <b>n</b> etwork <b>t</b> o <b>h</b> ost <b>s</b> hort |
| htonl    | <b>h</b> ost <b>t</b> o <b>n</b> etwork <b>l</b> ong  |
| htons    | <b>h</b> ost <b>t</b> o <b>n</b> etwork <b>s</b> hort |

- Beej's Guide to Network Programming
  - Example code: <https://github.com/wlfuh/bgreeves.sockets.starter/tree/master>
  - Feel free to copy-paste portions of this code for your Assignment 1 as necessary!
- Use existing documentation (i.e. man pages) – these are generally very complete!
  - Nice UI wrapper: <https://man7.org/linux/man-pages/>
- StackOverflow and ChatGPT can be very helpful for socket programming
  - Many of these things are formulaic; others likely have encountered similar problems
  - Understanding code instead of pure copy-paste can help prevent further questions down the line



```
1 #include <arpa/inet.h> // ntohl() et al.  
2 #include <netdb.h> // gethostbyname(), struct hostent  
3 #include <netinet/in.h> // struct sockaddr_in  
4 #include <sys/socket.h> // getsockname()  
5 #include <chrono> // std::chrono::*
```